

Lesson 19 Structured Note: Program for Evaluation of a Postfix Expression

0. Current Position in the Course

You are now in **Lesson 19** of the stack curriculum.

Before this lesson, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Bracket matching was extended to <code>()</code> , <code>[]</code> , and <code>{}</code> .
Lesson 13	You learned infix, prefix, postfix, precedence, and simple manual expression conversion.
Lesson 14	You learned associativity and unary operators.
Lesson 15	You learned the logic for converting infix to postfix using a stack.

Lesson	Main idea
Lesson 16	You wrote an infix-to-postfix program using an array character stack.
Lesson 17	You wrote an infix-to-postfix program using a linked-list character stack.
Lesson 18	You learned the theory and procedure of postfix evaluation.

Lesson 19 continues:

Phase 5: Postfix Evaluation and Integration

Lesson 18 explained the idea of postfix evaluation.

Lesson 19 explains how to write the **program** for postfix evaluation.

In simple words:

- Lesson 18 taught the rule.
- Lesson 19 teaches the code.

1. Lesson 19 Goal

By the end of this lesson, you should understand:

- What postfix evaluation means.
- Why postfix evaluation uses a stack.
- Why the stack stores integers during evaluation.
- How to scan a postfix expression from left to right.
- How to identify operands and operators.
- Why digit characters must be converted into integers.
- How `postfix[i] - '0'` works.
- Why we use variables `x1`, `x2`, and `r`.
- Why pop order matters.
- How to use a `switch` statement for arithmetic operators.
- How to write a complete C program for postfix evaluation.
- How to dry-run the program step by step.
- The time and space complexity of the program.
- Common beginner mistakes to avoid.

2. Quick Review: What Is a Postfix Expression?

An expression can be written in different forms.

The two forms important here are:

1. **Infix**
2. **Postfix**

2.1 Infix Notation

In infix notation, the operator is written **between** the operands.

Example:

3 + 5

Here:

Symbol	Role
3	Operand
+	Operator
5	Operand

So the structure is:

Operand Operator Operand

Humans usually write expressions in infix form.

2.2 Postfix Notation

In postfix notation, the operator is written **after** the operands.

Example:

35+

This means the same thing as:

3 + 5

The structure is:

Operand Operand Operator

So:

35+

means:

3 + 5

Result:

8

3. What Is Postfix Evaluation?

Postfix evaluation means calculating the final answer of a postfix expression.

Example:

35+

This postfix expression means:

3 + 5

Final answer:

8

Another example:

82/

This means:

8 / 2

Final answer:

4

4. Why Do We Use a Stack for Postfix Evaluation?

A postfix expression gives operands first and operators later.

Example:

35+

When we scan from left to right:

1. We see 3.
2. We do not know the operator yet, so we store 3.
3. We see 5.
4. We still do not know the operator yet, so we store 5.
5. We see +.
6. Now we can use the two most recent numbers.

A stack is perfect here because a stack follows:

LIFO = Last In, First Out

The most recent operands are needed first when an operator appears.

So the stack is used to temporarily store numbers.

5. Main Rule of Postfix Evaluation

When scanning a postfix expression from left to right:

Symbol type	Action
Operand	Push it onto the stack.
Operator	Pop two values, calculate, and push the result.

In simple pseudocode:

```
for each symbol in postfix expression:

    if symbol is an operand:
        push symbol onto the stack

    if symbol is an operator:
        x2 = pop()
        x1 = pop()
        r = x1 operator x2
        push r
```

At the end, the stack contains one value.

That value is the final answer.

6. Why the Stack Stores Integers in Lesson 19

In earlier lessons, the stack sometimes stored characters.

For example, during infix-to-postfix conversion, the stack stored operators like:

```
+ - * /
```

Those are characters.

But in postfix evaluation, we calculate arithmetic.

So the stack must store numbers such as:

3, 5, 15, 18

That means Lesson 19 uses an **integer stack**.

Important idea:

Task	Stack stores
Infix-to-postfix conversion	Operators as characters
Postfix evaluation	Numbers as integers

7. Operand and Operator in This Lesson

7.1 Operand

An operand is a number used in the expression.

In this beginner lesson, operands are usually single digits:

0 1 2 3 4 5 6 7 8 9

Examples:

3
5
8
2

When the program sees an operand, it pushes it onto the stack.

7.2 Operator

An operator performs an arithmetic operation.

The operators used in this lesson are:

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division

When the program sees an operator, it pops two values from the stack.

Then it calculates the result.

Then it pushes the result back onto the stack.

8. Very Important: Pop Order Matters

This is one of the most important points in Lesson 19.

When an operator is found, we pop two values.

The correct order is:

```
x2 = pop(&st); // right operand
x1 = pop(&st); // left operand
```

Then calculate:

```
r = x1 operator x2;
```

So the full pattern is:

```
x2 = pop(&st);
x1 = pop(&st);
r = x1 operator x2;
push(&st, r);
```

8.1 Why the First Popped Value Is `x2`

A stack removes the top value first.

In postfix evaluation, the top value is usually the **right side** of the operation.

Example:

82/

This means:

8 / 2

Dry run:

1. Push `8`.
2. Push `2`.
3. See `/`.
4. Pop first value: `2`.
5. Pop second value: `8`.

So:

```
x2 = 2;  
x1 = 8;  
r = x1 / x2;  
r = 8 / 2;  
r = 4;
```

Correct answer:

4

8.2 What Happens If We Reverse the Order?

Wrong code:

```
x1 = pop(&st);  
x2 = pop(&st);  
r = x1 / x2;
```

For:

82/

This would do:

2 / 8

That is wrong.

So always remember:

```
x2 = pop(&st);    // first pop  
x1 = pop(&st);    // second pop
```

8.3 Operators Affected by Pop Order

Addition and multiplication are not very sensitive to order:

```
3 + 5 = 5 + 3  
3 * 5 = 5 * 3
```

But subtraction and division are very sensitive:

```
3 - 5 is not the same as 5 - 3  
8 / 2 is not the same as 2 / 8
```

So we must always use the correct pop order.

9. Character Digits vs Integer Values

This is another very important part of Lesson 19.

In C, a postfix expression is stored as a string.

Example:

```
char postfix[] = "35+";
```

Inside this string:

```
'3'  
'5'  
'+'
```

The value `'3'` is a character.

It is not the integer `3`.

This difference matters because we want to calculate arithmetic.

10. Why We Need `postfix[i] - '0'`

Suppose we write:

```
push(&st, postfix[i]);
```

If `postfix[i]` is `'3'`, this pushes the character code of `'3'`, not the integer `3`.

That is not what we want.

We want to push the number `3`.

So we write:

```
push(&st, postfix[i] - '0');
```

10.1 How `postfix[i] - '0'` Works

In C, digit characters are stored in order:

```
'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'
```

So subtracting `'0'` converts a digit character into its integer value.

Examples:

```
'3' - '0' = 3  
'5' - '0' = 5  
'8' - '0' = 8  
'0' - '0' = 0
```

So:

```
postfix[i] - '0'
```

means:

```
Convert this digit character into a real integer.
```

11. Example of ASCII Conversion

Suppose:

```
postfix[i] = '7';
```

If we push directly:

```
push(&st, postfix[i]);
```

we are not pushing integer `7`.

We are pushing the character code of `'7'`.

Correct:

```
push(&st, postfix[i] - '0');
```

This pushes:

7

Simple rule:

Digit character - '0' = integer digit

12. Stack Structure Used in Lesson 19

Since we are evaluating numbers, the stack stores integers.

So the stack structure is:

```
struct Stack {  
    int size;  
    int top;  
    int *S;  
};
```

Meaning of each field:

Field	Meaning
size	Maximum number of integers the stack can store.
top	Index of the current top element.
S	Pointer to the integer array used as the stack.

Example:

```
Stack  
├─ size = 9  
├─ top = -1  
└─ S -> integer array
```

13. Empty and Full Conditions

Because this is an array stack, the same array stack conditions apply.

13.1 Empty Condition

The stack is empty when:

```
top == -1
```

This means no active element exists in the stack.

Important:

```
top = -1
```

does not mean:

```
S[-1]
```

Array index `-1` is invalid.

`-1` is only a marker meaning:

```
The stack is empty.
```

13.2 Full Condition

The stack is full when:

```
top == size - 1
```

Example:

If:

```
size = 5
```

Valid indices are:

```
0, 1, 2, 3, 4
```

The last valid index is:

```
size - 1 = 4
```

So the stack is full when:

```
top == 4
```

14. Helper Function: `isEmpty()`

Meaning

`isEmpty()` checks whether the stack has no active elements.

Code:

```
int isEmpty(struct Stack st) {  
    return st.top == -1;  
}
```

Return values:

Return value	Meaning
1	Stack is empty.
0	Stack is not empty.

Example:

If:

```
top = -1
```

Then:

```
isEmpty(st)
```

returns:

```
1
```

15. Helper Function: `isFull()`

Meaning

`isFull()` checks whether the stack has reached its maximum capacity.

Code:

```
int isFull(struct Stack st) {  
    return st.top == st.size - 1;  
}
```

Return values:

Return value	Meaning
1	Stack is full.
0	Stack is not full.

Example:

If:

```
size = 5  
top = 4
```

Check:


```
top == size - 1  
4 == 5 - 1  
4 == 4
```

This is true.

So `isFull(st)` returns:

1

16. Push Operation

Meaning

`push()` inserts a value at the top of the stack.

For postfix evaluation, we push integer values.

Example:

```
push(&st, 3);  
push(&st, 5);
```

The stack becomes:

3, 5

Here, `5` is at the top.

Push Code

```
void push(struct Stack *st, int x) {  
    if (isFull(*st)) {  
        printf("Stack Overflow\n");  
    } else {  
        st->top++;  
        st->S[st->top] = x;  
    }  
}
```

```
}  
}
```

Push Explanation

A successful push does two things:

1. Increase `top` by `1`.
2. Store the new value at `S[top]`.

Example:

Before push:

```
top = -1
```

Call:

```
push(&st, 3);
```

Step 1:

```
top: -1 -> 0
```

Step 2:

```
S[0] = 3
```

Now the stack contains:

```
3
```

17. Pop Operation

Meaning

`pop()` removes and returns the top value from the stack.

Example:

Before pop:

3, 5

The top value is:

5

After pop:

3

Returned value:

5

Pop Code

```
int pop(struct Stack *st) {
    int x = -1;

    if (isEmpty(*st)) {
        printf("Stack Underflow\n");
    } else {
        x = st->S[st->top];
        st->top--;
    }

    return x;
}
```

Pop Explanation

A successful pop does three things:

1. Save the value at `S[top]`.
2. Decrease `top` by `1`.
3. Return the saved value.

Example:

Before pop:

```
S[0] = 3  
S[1] = 5  
top = 1
```

Call:

```
pop(&st);
```

Step 1:

```
x = S[top]  
x = S[1]  
x = 5
```

Step 2:

```
top: 1 -> 0
```

Step 3:

```
return 5
```

18. Helper Function: `isOperand()`

Meaning

`isOperand()` checks whether a character is a digit operand.

In this beginner version, operands are single digits from `0` to `9`.

Code:

```
int isOperand(char x) {  
    if (x >= '0' && x <= '9')  
        return 1;  
    return 0;  
}
```

Explanation

The condition:

```
x >= '0' && x <= '9'
```

checks whether `x` is one of these characters:

```
'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'
```

If yes, the function returns:

```
1
```

If no, it returns:

```
0
```

Example:

```
isOperand('3') returns 1  
isOperand('8') returns 1  
isOperand('+') returns 0  
isOperand('*') returns 0
```

19. Main Function: `evaluatePostfix()`

The main function for this lesson is:

```
int evaluatePostfix(char *postfix)
```

It receives a postfix expression as a string.

Example:

```
"35*62/+4-"
```

It returns the final integer result.

Example:

```
14
```

20. Steps Inside `evaluatePostfix()`

The function does these main steps:

1. Create an integer stack.
2. Set stack size equal to the length of the postfix expression.
3. Initialize `top` to `-1`.
4. Allocate memory for the stack array.
5. Scan the postfix expression from left to right.
6. If the symbol is an operand, convert it to integer and push it.
7. If the symbol is an operator, pop two values.
8. Use a `switch` statement to calculate the result.
9. Push the result back onto the stack.
10. After the loop ends, pop the final answer.

11. Free the stack memory.
 12. Return the answer.
-

21. Creating the Stack Inside `evaluatePostfix()`

Code:

```
struct Stack st;  
st.size = strlen(postfix);  
st.top = -1;  
st.S = (int *)malloc(st.size * sizeof(int));
```

21.1 Why Use `strlen(postfix)`?

`strlen(postfix)` gives the number of characters in the postfix expression.

Example:

```
char postfix[] = "35*62/+4-";
```

This expression has 9 characters.

So:

```
strlen(postfix)
```

returns:

```
9
```

We use this as the stack size because the stack will never need to store more values than the number of expression symbols.

21.2 Why Set `top = -1`?

At the beginning, the stack is empty.

For an array stack, empty means:

```
top = -1
```

So we write:

```
st.top = -1;
```

21.3 Why Use `malloc()` ?

The stack array is created dynamically.

Code:

```
st.S = (int *)malloc(st.size * sizeof(int));
```

Meaning:

```
Create enough memory to store st.size integers.
```

Example:

If:

```
st.size = 9
```

Then `malloc()` creates memory for 9 integers.

22. Variables `x1`, `x2`, and `r`

Inside the function, we use:

```
int x1, x2, r;
```

Meaning:

Variable	Meaning
x2	First popped value; right operand.
x1	Second popped value; left operand.
r	Result after applying the operator.

Example:

For:

82/

When / is found:

```
x2 = pop(&st);    // 2
x1 = pop(&st);    // 8
r = x1 / x2;      // 8 / 2 = 4
push(&st, r);     // push 4
```

23. The Scanning Loop

The program scans the postfix expression using a loop:

```
for (i = 0; postfix[i] != '\0'; i++) {
    // process postfix[i]
}
```

23.1 What Does postfix[i] != '\0' Mean?

In C, a string ends with a special character:

'\0'

This is called the null terminator.

The loop continues until it reaches the end of the string.

Example:

```
"35+"
```

is stored like this:

```
'3' '5' '+' '\0'
```

The loop processes:

```
'3'  
'5'  
'+'
```

It stops at:

```
'\0'
```

24. Operand Case in the Loop

If the current symbol is an operand, the program pushes it.

Code:

```
if (isOperand(postfix[i])) {  
    push(&st, postfix[i] - '0');  
}
```

Example:

If:

```
postfix[i] = '3'
```

Then:

```
postfix[i] - '0'
```

becomes:

```
3
```

Then the program pushes integer `3` into the stack.

25. Operator Case in the Loop

If the current symbol is not an operand, it is treated as an operator.

Code:

```
else {
    x2 = pop(&st);
    x1 = pop(&st);

    switch (postfix[i]) {
        case '+':
            r = x1 + x2;
            break;

        case '-':
            r = x1 - x2;
            break;

        case '*':
            r = x1 * x2;
            break;

        case '/':
            r = x1 / x2;
            break;
    }

    push(&st, r);
}
```

This is the heart of postfix evaluation.

26. Why Use a `switch` Statement?

When the program sees an operator, it must decide which arithmetic operation to perform.

The operator may be:

```
+  
-  
*  
/
```

A `switch` statement is a clean way to choose the correct operation.

26.1 Switch for Addition

```
case '+':  
    r = x1 + x2;  
    break;
```

If the operator is `+`, add the two values.

26.2 Switch for Subtraction

```
case '-':  
    r = x1 - x2;  
    break;
```

If the operator is `-`, subtract the right operand from the left operand.

Important:

```
x1 - x2
```

not:

```
x2 - x1
```

26.3 Switch for Multiplication

```
case '*':  
    r = x1 * x2;  
    break;
```

If the operator is `*`, multiply the two values.

26.4 Switch for Division

```
case '/':  
    r = x1 / x2;  
    break;
```

If the operator is `/`, divide the left operand by the right operand.

Important:

$x1 / x2$

not:

$x2 / x1$

27. Why Push the Result Back?

After calculating the result, the result may be used by another operator later.

Example:

$35 * 62 / +4 -$

At the beginning:

$3 \ 5 \ *$

means:

```
3 * 5 = 15
```

But 15 is not the final answer yet.

It must be used later.

So we push 15 back onto the stack.

Code:

```
push(&st, r);
```

Simple meaning:

```
Store the partial answer for later use.
```

28. Getting the Final Answer

After the loop ends, the stack should contain only one value.

That value is the final answer.

Code:

```
int answer = pop(&st);  
free(st.S);  
return answer;
```

28.1 Why Pop at the End?

The final answer is at the top of the stack.

So we pop it and return it.

Example:

Final stack:

14

Then:

```
answer = pop(&st);
```

returns:

14

28.2 Why Use `free(st.S)` ?

The stack array was created using `malloc()` .

So after using it, we should release the memory.

Code:

```
free(st.S);
```

This prevents memory waste.

29. Complete Beginner C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Stack {
    int size;
    int top;
    int *S;
};

int isEmpty(struct Stack st) {
```

```

        return st.top == -1;
    }

    int isFull(struct Stack st) {
        return st.top == st.size - 1;
    }

    void push(struct Stack *st, int x) {
        if (isFull(*st)) {
            printf("Stack Overflow\n");
        } else {
            st->top++;
            st->S[st->top] = x;
        }
    }

    int pop(struct Stack *st) {
        int x = -1;

        if (isEmpty(*st)) {
            printf("Stack Underflow\n");
        } else {
            x = st->S[st->top];
            st->top--;
        }

        return x;
    }

    int isOperand(char x) {
        if (x >= '0' && x <= '9')
            return 1;
        return 0;
    }

    int evaluatePostfix(char *postfix) {
        struct Stack st;
        st.size = strlen(postfix);
        st.top = -1;
        st.S = (int *)malloc(st.size * sizeof(int));

        int i;
        int x1, x2, r;

        for (i = 0; postfix[i] != '\0'; i++) {
            if (isOperand(postfix[i])) {
                push(&st, postfix[i] - '0');
            } else {

```



```

        x2 = pop(&st);
        x1 = pop(&st);

        switch (postfix[i]) {
            case '+':
                r = x1 + x2;
                break;

            case '-':
                r = x1 - x2;
                break;

            case '*':
                r = x1 * x2;
                break;

            case '/':
                r = x1 / x2;
                break;
        }

        push(&st, r);
    }
}

int answer = pop(&st);
free(st.S);

return answer;
}

int main() {
    char postfix[] = "35*62/+4-";

    int result = evaluatePostfix(postfix);

    printf("Result = %d\n", result);

    return 0;
}

```

30. Expected Output

For:

```
char postfix[] = "35*62/+4-";
```

The output is:

Result = 14

31. Dry Run 1: Simple Expression 35+

Expression:

35+

Meaning:

3 + 5

Initial stack:

empty

Trace:

Step	Symbol	Action	Stack after action
1	3	Operand, push 3	3
2	5	Operand, push 5	3, 5
3	+	Pop 5, pop 3, calculate $3 + 5 = 8$, push 8	8

Final stack:

8

Final answer:

8

32. Dry Run 2: Division Expression 82/

Expression:

82/

Meaning:

8 / 2

Initial stack:

empty

Trace:

Step	Symbol	Action	Stack after action
1	8	Operand, push 8	8
2	2	Operand, push 2	8, 2
3	/	Pop 2, pop 8, calculate $8 / 2 = 4$, push 4	4

Final answer:

4

Important lesson:

```
x2 = 2
x1 = 8
r = x1 / x2
r = 8 / 2
r = 4
```

33. Full Required Dry Run: 35*62/+4-

Expression:

35*62/+4-

We scan from left to right.

Initial stack:

empty

33.1 Step-by-Step Trace

Step	Symbol	Action	Stack after action
1	3	Push 3	3
2	5	Push 5	3, 5
3	*	Pop 5, pop 3, calculate $3 * 5 = 15$, push 15	15
4	6	Push 6	15, 6
5	2	Push 2	15, 6, 2
6	/	Pop 2, pop 6, calculate $6 / 2 = 3$, push 3	15, 3
7	+	Pop 3, pop 15, calculate $15 + 3 = 18$, push 18	18
8	4	Push 4	18, 4
9	-	Pop 4, pop 18, calculate $18 - 4 = 14$, push 14	14

Final stack:

14

Final answer:

14

34. Understanding the Full Expression in Parts

Expression:

35*62/+4-

Break it into meaningful operations:

3 5 *	-> 3 * 5 = 15
6 2 /	-> 6 / 2 = 3
15 3 +	-> 15 + 3 = 18
18 4 -	-> 18 - 4 = 14

So the final answer is:

14

35. Why We Do Not Need Precedence During Postfix Evaluation

In infix notation, precedence matters.

Example:

3 + 5 * 2

Multiplication happens before addition.

But in postfix notation, the order is already decided.

Example postfix:

352*+

The postfix expression already tells us:

1. First do 5 * 2 .

2. Then add `3`.

So during postfix evaluation, we do **not** check precedence.

We simply scan from left to right.

Important idea:

Precedence is handled during conversion.
Postfix evaluation only follows the postfix order.

36. Beginner Limitations of This Program

This beginner program is useful, but it has limits.

36.1 It Handles Single-Digit Operands

It works for expressions like:

35+
82/
35*62/+4-

But it does not properly handle multi-digit numbers like:

12 34 +

Why?

Because it scans one character at a time.

So `12` would be read as:

'1'
'2'

not as:

12

36.2 It Assumes the Expression Is Valid

The beginner program assumes the postfix expression is correct.

For example, it assumes operators always have enough operands to pop.

Invalid expression:

3+

This is invalid because `+` needs two operands, but only `3` exists.

A stronger program would check for this more carefully.

36.3 It Does Not Handle Spaces

This beginner version does not handle spaces like:

3 5 +

It expects:

35+

36.4 It Does Not Handle Negative Numbers Directly

An expression containing negative numbers needs more advanced parsing.

Example:

-3 5 +

This beginner version is not designed for that.

37. Common Beginner Mistakes

Mistake 1: Pushing the Character Instead of the Integer

Wrong:

```
push(&st, postfix[i]);
```

Correct:

```
push(&st, postfix[i] - '0');
```

Why?

Because `'3'` is a character, but `3` is an integer.

Mistake 2: Reversing Pop Order

Wrong:

```
x1 = pop(&st);  
x2 = pop(&st);  
r = x1 - x2;
```

Correct:

```
x2 = pop(&st);  
x1 = pop(&st);  
r = x1 - x2;
```

The first popped value is the right operand.

The second popped value is the left operand.

Mistake 3: Using Precedence During Evaluation

Wrong idea:

When evaluating postfix, check whether * has higher precedence than +.

Correct idea:

Postfix already contains the correct order.
Just scan from left to right.

Mistake 4: Forgetting to Push the Result Back

Wrong:

```
r = x1 + x2;
```

Correct:

```
r = x1 + x2;  
push(&st, r);
```

The result must be pushed back because it may be used later.

Mistake 5: Forgetting the Final Pop

At the end, the final answer is on the stack.

So we need:

```
answer = pop(&st);
```

If we do not pop the final value, we do not return the result.

Mistake 6: Forgetting to Free Memory

Since we used `malloc()`, we should use `free()`.

Correct:

```
free(st.S);
```

This releases the stack memory.

38. Time Complexity

Let n be the number of symbols in the postfix expression.

The program scans each symbol once.

Each push and pop operation takes constant time:

```
O(1)
```

So the total time complexity is:

```
O(n)
```

Simple meaning:

```
If the expression has n characters, the loop runs n times.
```

39. Space Complexity

The stack may store several operands while evaluating the expression.

In the worst case, many operands may appear before an operator.

Example:

```
12345++++
```

Several numbers may be pushed before calculations reduce them.

So the stack may need space proportional to n .

Space complexity:

$O(n)$

40. Lesson 19 Algorithm Summary

The full algorithm is:

```
Create an integer stack.
Scan the postfix expression from left to right.

For each symbol:

    If the symbol is a digit:
        Convert it to integer using symbol - '0'.
        Push it onto the stack.

    Otherwise, the symbol is an operator:
        x2 = pop()
        x1 = pop()

        Use switch:
            + means x1 + x2
            - means x1 - x2
            * means x1 * x2
            / means x1 / x2

        Push the result back onto the stack.

After the scan finishes:
    Pop and return the final answer.
```

41. Full Code Logic in Simple Words

This line:

```
if (isOperand(postfix[i]))
```

means:

Is the current symbol a number?

This line:

```
push(&st, postfix[i] - '0');
```

means:

Convert the digit character to an integer and push it.

These lines:

```
x2 = pop(&st);  
x1 = pop(&st);
```

mean:

Take the two most recent numbers from the stack.

This line:

```
switch (postfix[i])
```

means:

Check which operator we found.

This line:

```
push(&st, r);
```

means:

Save the partial answer back into the stack.

This line:

```
return answer;
```

means:

Return the final evaluated value.

42. Final Revision Table

Concept	Key idea
Postfix evaluation	Calculate the result of a postfix expression.
Stack role	Stores operands and partial results.
Operand	Push it onto the stack.
Operator	Pop two values, calculate, push result.
x2	First popped value; right operand.
x1	Second popped value; left operand.
r	Result of calculation.
postfix[i] - '0'	Converts digit character to integer.
switch	Chooses the correct arithmetic operation.
Final answer	The last value left on the stack.
Time complexity	$O(n)$
Space complexity	$O(n)$

43. Practice Questions

Practice 1

Evaluate:

23+

Expected idea:

$$2 + 3 = 5$$

Practice 2

Evaluate:

$$82 -$$

Expected idea:

$$8 - 2 = 6$$

Practice 3

Evaluate:

$$93 /$$

Expected idea:

$$9 / 3 = 3$$

Practice 4

Evaluate:

$$23 * 54 * +$$

Breakdown:

```
2 * 3 = 6
5 * 4 = 20
6 + 20 = 26
```

Expected answer:

26

44. Beginner Checkpoints

You understand Lesson 19 if you can answer these questions:

1. Why does postfix evaluation use a stack?
2. What happens when the current symbol is an operand?
3. What happens when the current symbol is an operator?
4. Why do we write `postfix[i] - '0'`?
5. Why is `x2` popped before `x1`?
6. Why is subtraction affected by pop order?
7. Why is division affected by pop order?
8. What does the `switch` statement do?
9. Why do we push the result back onto the stack?
10. What is the final value left in the stack?
11. What is the time complexity?
12. What is the space complexity?

45. Lesson 19 Key Sentence

You understand Lesson 19 if you can explain this:

In a postfix evaluation program, digit characters are converted using `- '0'`, operands are pushed into an integer stack, operators pop `x2` first and `x1` second, a `switch` computes the result, and the result is pushed back until one final answer remains.

46. Final Mental Model

Think of the stack as a small working table.

When you see a number:

Put it on the table.

When you see an operator:

Take the top two numbers from the table.
Apply the operator.
Put the result back on the table.

At the end:

Only one number remains.
That number is the answer.

That is postfix evaluation.